Haoyang Tang

tanghy@idsse.ac.cn | Sanya, Hainan, China | github.com/wojiaonioo | wojiaonioo.github.io

M.S. student in physical oceanography | Differentiable ocean models, GPU acceleration, variational data assimilation and hybrid modeling

Education

Institute of Deep-sea Science and Engineering, Chinese Academy of Sciences **Physical Oceanography, M.S.** 2025.09 - present

Lanzhou University **Atmospheric Sciences, B.S.** 2019.09 - 2023.06

Project

DEFgcm: a differentiable GPU ocean model (sole developer) 2026.03 - present

MITgcm, a widely used ocean model, computes its gradients with Fortran adjoint code on CPUs, so it cannot be trained together with large neural networks on GPUs. DEFgcm reimplements the MITgcm discretizations in PyTorch, and one program computes both the integration and its gradient on GPUs, for 4D-Var data assimilation and online training of neural networks.

Model development and validation

- Implemented the hydrostatic and nonhydrostatic Boussinesq equations on latitude–longitude and cubed-sphere grids, with KPP vertical mixing, a pressure-dependent equation of state, an implicit free surface and open boundaries.
- Matched MITgcm in temperature, salinity, velocity and SSH over 29-day South China Sea runs on 960×912×90 and 160×152×90 grids, to within the difference between two MITgcm runs that change only the summation order (RMS temperature difference on the 960×912×90 grid: $\mathbf{3.6\times10^{-5}}$ **K** against $\mathbf{4.0\times10^{-5}}$ **K**).

GPU acceleration

- Ran faster than MITgcm on every grid from 480×480×90 up with four A800 GPUs, and **1.6**× as fast as MITgcm on 768 CPU cores on a 960×960×90 grid (77 ms per step).
- Ran the nonhydrostatic configuration **3.5**× as fast as MITgcm on 640 CPU cores with four A800 GPUs.
- Fused the tensor operations into GPU kernels with torch.compile, and wrote CUDA kernels for the vertical solves, equation of state, mixing and advection, with backward kernels for some of them.
- Implemented multi-GPU domain decomposition and its backward pass, so grids too large for one GPU can be differentiated across several; four A800 GPUs integrate a 2400×2400×90 grid.
- Reached a 1.8–1.9× speedup for each doubling of the GPU count on grids from 960×960×90 up.
- Used segment checkpointing with recomputation, and moved the states that exceed GPU memory to host memory; a gradient costs 2.9 to 5.3 times a forward integration.

Variational data assimilation and hybrid modeling

- Built 4D-Var on the model gradient; over 79 three-day windows of South China Sea twin experiments, it lowered the forecast error three days after the window by **40%** on average.
- Trained a neural network online through the same gradient to produce initial states, without precomputed analyses as labels.
- On 13 test windows after the training period, the network recovered **81%** of the 4D-Var improvement from one gradient evaluation, against 44% for one gradient step at the same cost.
- 4D-Var needs about 11 evaluations of J and its gradient to reach the error of the network, which needs only one.
- Differentiated a neural network embedded inside the model time step: its weight gradients match finite differences to a relative 10^{-6}, and it adds only 8% to the time of a gradient evaluation.

Skills

Programming: PyTorch (torch.compile, custom operators, autograd), CUDA C++, Python, Fortran, NCCL / MPI
Scientific computing: ocean numerical models, automatic differentiation and adjoints, variational data assimilation, hybrid modeling
Tools: Git, SLURM, Nsight, Claude Code, Codex